TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE PINOTEPA
Departamento de Ingenierías
C.C.T. 20DIT0005K

Investigación en Libreta de la Unidad 2

PRESENTA:
Álvaro Hernández Carmona

Maestro:
Nieto Vásquez Carlos Manuel

SÉPTIMO SEMESTRE

INGENIERÍA EN SISTEMAS COMPUTACIONALES (I.S.C.)

PERÍODO AGOSTO / DICIEMBRE 2026

PINOTEPA NACIONAL, OAX. **SEPTIEMBRE, 2026.**

Modelo de Programacion Funcional 28 09 26

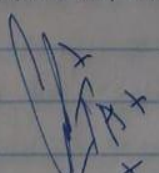
El modelo de programacion funcional es un paradigma de programacion que trata los programas principalmente como una convinacion de funciones que reciben datos y producen resultados, evitando modificar datos o mantener estados que cambien durante la ejecucion

A diferencia de la programacion imperativa, donde normalmente indicamos como realizar una tarea paso a paso, en la programacion funcional se busca expresar que resultado queremos obtener mediante funciones.

Caracteristicas Principales

- Funciones: son el elemento central del programa. Una funcion puede recibir datos como entrada y devolver un resultado

- Inmutabilidad: se procura no modificar los datos existentes, en lugar de cambiarlos, se generan nuevos valores



- Funciones puras: una funcion pura, para una misma entrada, siempre produce el mismo resultado y no provoca efectos secundarios

- Composicion: Varios funciones pueden convinarse para resolver problemas mas complejos

tienen su origen en este concepto

Aplicacion de Funciones

Consiste en darle un argumento a una funcion

$$(\lambda x.\ x * 2)\ 5$$

Se sustituye x por 5: 5 x 2
Resultado: 10

Este proceso se conoce como reduccion beta

Reduccion Beta

Es una de las operaciones fundamentales del calculo lambda

Ejemplo: $(\lambda x.\ x+3)\ 7$

Primero sustituimos: 7+3

Y finalmente: 10

Formalmente: $(\lambda x.\ M)\ N \rightarrow M[x := N]$

Es decir, se reemplaza la variable x por N dentro de M

Funciones de varias variables

Tambien podemos representar funciones que reciben varios argumentos.

Por ejemplo:

$$\lambda x.\ \lambda y.\ x + y$$

Equivale conceptualmente a:

$$f(x, y) = x + y$$

Podemos aplicarla:

$$((\lambda x.\ \lambda y.\ x + y)\ 5)\ 3$$

Primero:

$$\lambda y.\ 5 + y$$

Despues:

$$5 + 3$$

Resultado:

$$8$$

Esto esta relacionado con la currificacion (curry; ing), un concepto muy importante en programacion funcional

Variables libres y ligadas

Este concepto es importante cuando empieces a estudiar formalmente calculo lambda:

En: $\lambda x. x + y$

x esta ligada porque pertenece a la funcion λx

Mientras que y esta libre, porque no fue definida por ninguna abstraccion lambda

Funciones Recursivas

Las funciones recursivas son un concepto fundamental de la programacion funcional. Una funcion es recursiva cuando se llama a si misma para resolver un problema, dividiendolo en problemas mas pequenos del mismo tipo. Este proceso continua hasta llegar a una condicion que detiene la recurcion.

La recursividad es especialmente importante en la programacion funcional por que permite trabajar con estructuras como listas, arboles y expreciones matematicas sin depender necesariamente de ciclos tradicionales como for o while

¿Como funciona una funcion recursiva?

Una funcion recursiva normalmente tiene dos partes

- Caso base: condicion que detiene la ejecucion de la funcion
- Caso recursivo: parte donde la funcion se vuelve a llamar asi misma, utilizando un problema mas pequeño

Por ejemplo: para calcular el factorial de un numero

$$n! = n \times (n-1) \times (n-2) \times \ldots \times 1$$

El factorial puede definirse de forma recursiva como:

$$factorial(n) = \begin{cases} 1 & si\ n = 0 \\ n \times factorial(n-1) & si\ n > 0 \end{cases}$$

si calculamos factorial (5), la computadora realiza aproximadamente

factorial (5)
→ 5 x factorial (4)
→ 5 x 4 x factorial (3)
→ 5 x 4 x 3 factorial (2)
→ 5 x 4 x 3 x 2 factorial (1)
→ 5 x 4 x 3 x 2 x 1
→ 120

El caso base es factorial (0) = 1. Sin este caso, la funcion continuara llamandose indefinidamente

- Funciones de orden superior: Una funcion puede recibir otra funcion como argumento o devolver una funcion como resultado
- Recursividad: se utiliza frecuentemente en lugar de estructuras repetitivas tradicionales como for o while

Calculo Lambda de Alonzo Church

28 09 26

Es un sistema formal de la logica matematica que sirve para representar y estudiar funciones y su evaluacion. Fue desarrollado por Alonzo Church en la decada de 1930 y es una de los fundamentos teoricos de la programacion funcional.

Su forma basica es:

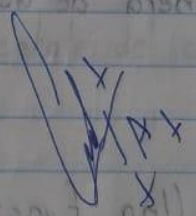
$$\lambda x. x+1$$

Se lee: "lambda x, x mas 1". Significa una funcion que recibe x y devuelve x+1. Por ejemplo

$$(\lambda x. x+1)5$$

Se sustituye x por 5

$$5+1=6$$



Por eso, el calculo lambda esta directamente relacionado con lenguajes como Lisp, Scheme, Racket y Haskell, y conceptos actuales como los funciones anonimos (lambda), la composicion de funciones y las funciones de orden superior.

El calculo lambda es un modelo matematico de la computacion. Church buscaba una forma formal de representar que es una funcion y como puede ejecutarse sin depender de un lenguaje de programacion especifica

Todo se construye principalmente con tres elementos:

1.- Variables: x, y, z

2.- Abstraccion: crear una funcion

$$\lambda x.\ x+z$$

3.- Aplicacion: utilizar una funcion

$$(\lambda x.\ x+1)\ 5$$

Abstraccion lambda

Es la manera de definir una funcion:

$$\lambda x.\ x*2$$

Significa: Una funcion que recibe x y devuelve $x*2$

En un lenguaje Racket:

```
(lambda (x) (* x 2))
```

Y en Phyton:

```
lambda x: x*2
```

Por eso las expreciones lambda que se utilizan en Racket

Erlang y Elixir

Erlang

Erlang es un lenguaje de programacion funcional creada en la decada de 1980 por Ericsson, fue diseñado originalmente para desarrollar sistemas de telecomunicaciones que necesitaban funcionar de manera continua, confiable y tolerante a fallos

Una de sus principales caracteristicas es que permite trabajar con muchos procesos de manera simultanea. Estos procesos pueden comunicarse entre si mediante el envio de mensajes, lo que hace que Erlang sea especialmente adecuada para sistemas distribuidos y aplicaciones que manejan grandes cantidades de conexiones.

Entre sus caracteristicas principales se encuentran:

- Programacion funcional
- Concurrencia y paralelismo
- Sistemas distribuidos
- Tolerancia a fallas
- Alta disponibilidad
- Capacidad para manejar miles o millones de procesos simultaneamente

Elixir

Elixir es un lenguaje de programacion funcional creado por Jose Valim y presentado en 2011. Esta construido sobre la BEAM, la maquina virtual utilizada por Erlang.

Elixir aprovecha muchas de las caracteristicas de Eatang, pero progarciona una sintaxis mas moderna y sensilla para muchos programadores. Tambien permite desarrollar aplicaciones concurrentes, distribuidas y tolerantes a fallos.

Elixir se utiliza principalmente para:

- Aplicaciones web
- Sistemas distribuidos
- Servicios en tiempo real
- Aplicaciones que necesitan muchas conexiones simultaneas
- Sistemas de telecomunicaciones

¿Donde se utiliza Erlang y Elixir en WhatsApp?

- Erlang: WhatsApp utiliza erlang principalmente en su infraestructura de servidores, especialmente para manejar millones de usuarios y conexiones simultaneas. Tambien permitio gestionar el envio y recepcion de mensajes, la comunicacion entre procesos y la distribucion de tareas.

- Elixir: No es el lenguaje principal utilizado por Whatsapp. Elixir utiliza la misma maquina virtual de Erlang, llamada BEAM, por lo que comparte caracteristicas como la concurrencia, distribucion y tolerancia a fallas

Funciones de Orden Superior

Las funciones de orden superior son aquellas que reciben otras funciones como argumentos, develven funciones como resultado o realizan ambas operaciones. Son una caracteristica importante de la programacion funcional, ya que permite reutilizar codigo, reducir instrucciones repetitivas y procesar datos de manera mas clara.

En el trabajo con listas, las funciones map, filter y fold/reduce permite transformar, seleccionar y combinar elementos sin necesidad de utilizar explicitamente ciclos tradicionales.

Funcion map

La funcion map aplica una funcion determinada a cada elemento de una lista con los resultados obtenidos Mantiene la cantidad de elementos de la lista original, siempre que la funcion aplicada produzca un resultado por cada elemento.

Ejemplo en Haskell:

```
main :: IO ()
main =print (map(*2) [1,2,3,4,5])
```

Resultado: [2,4,6,8,10]

Este ejemplo, map multiplica cada elemento de la lista por dos

El numero 1 se convierte en 2, el 2 en 4 y asi sucesivamente

Funcion filter

La funcion filter permite seleccionar los elementos de una lista que cumple una condicion determinada. Para ello, utiliza una funcion que devuelve un valor logico (True o False). Los elementos que cumplen la condicion se conservan, mientras que los demas se descartan

Caracteristica	Funcion Prefija	Funcion Infija	Funcion Postfija
Definicion	El operador se coloca antes de los operandos	El operador se coloca entre los operandos	El operador se coloca despues de los operandos
Estructura	Operador + operandos	Operador + operador + operando	operandos + operador
Ejemplo de suma	+53	5+3	53+
Uso de parentesis	Generalmente, no los necesita	Puede necesitarlos para establecer la prioridad de las operaciones	Generalmente, no los necesita
Orden de evaluacion	Se identifica primero el operador y despues sus operandos	Depende de la precedencia y asociatividad de los operadores	Los operadores se aplican a los operandos anteriores
Ventaja principal	Permite representar expresiones sin depender de parentesis	Es la notacion mas habitual en matematicas	Facilita la evaluacion mediante pilas de programacion
Aplicaciones	Logica matematica, Lisp y algunos lenguajes funcionales	Matematicas y la mayoria de los lenguajes de programacion	Calculadoras de pila, compiladores y maquinas virtuales
Ejemplo en Haskell	(+) 5 3	5+3	53+

Intervalos, Operadores y Evaluacion perezosa

Intervalos

Los intervalos permiten generar listas de valores consecutivas o siguiendo una secuencia determinada, sin necesidad de escribir cada elemento individualmente

En Haskell se utiliza la sintaxis [..]

Ejemplo: [1..10]

Genera una lista de numeros del 1 al 10;

Ejemplo: [2,4..12]

Genera una lista de numeros pares del 2 al 12

Operadores

Los operadores son simbolos o funciones que permiten realizar operaciones sobre valores y expresiones.

En Haskell se clasifican principalmente en:

- Aritmeticos: +, -, *, /, ^,
- Relacionales: ==, /=, <, >, <=, >=
- Lógicos: && (AND), || (OR), not (NOT)

• De listas: : (agrega un elemento al inicial, ++ (concatena listas), !! (accede a una posicion)

Ejemplos:

5+3 -- Resultado: 8
5>3 -- Resultado: True
True && False -- Resultado: False
[1,2] ++ [3,4] -- Resultado: [1,2,3,4]

Evaluacion Perezosa (Lazy Evaluation)

Es una caracteristica de Haskell que consiste en evaluar las expreciones unicamente cuando su resultado es necesario.

Esto permite trabajar con listas infinitas sin calcular todos sus elementos desde el principio, ya que Haskell obtiene solamente los valores que necesita

Ejemplo: take 5 [1..]

Resultado: [1,2,3,4,5]

Aunque [1..] representa una lista infinita, Haskell solamente evalua los primeros cinco elementos solicitados